"Empowerment through quality technical education"
AJEENKYA
DY Patil School of Engineering
Approved by AICTE, Recognized by Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University

Department of Computer Engineering

A Project Review Presentation on

AION : Autonomous Incident Observation & Navigation

An AI-Powered Incident Observation and Remediation Platform using Log Analysis, Git Correlation and Large Language Models for Root-Cause Analysis, Validated Code Fixes and Human-Approved Deployment

Presented by :
Aditya Bheke – 330114
Harshal Bhogawade – 330115
Malhar Bhoi – 330116

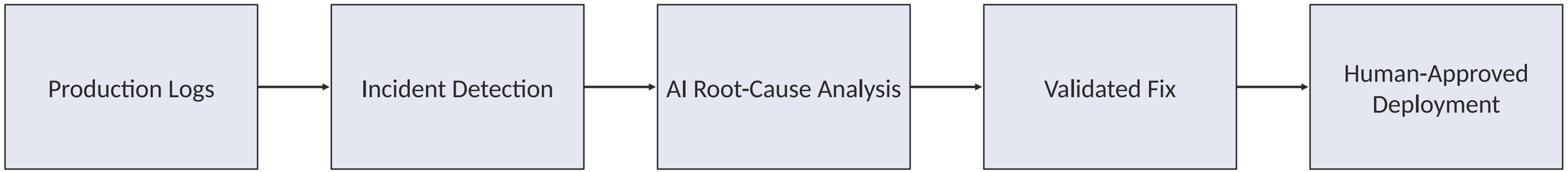
Under the guidance of : Dr. Yogesh Bahendwar

Introduction

Why AION?

- In conventional incident management, engineers manually inspect large volumes of logs during an outage.
- Tracing a failure to the recent code change that caused it is slow and depends on experienced engineers.
- Observability tools detect errors but stop there; writing, testing and deploying the fix remain separate manual steps.
- AION connects detection, Git correlation, AI root-cause analysis, patch generation and validation in one auditable workflow, with a human approving every production deployment.

Core Idea



Problem Statement

To design and develop an AI-powered platform that detects production incidents from application logs, traces them to the recent code change that caused them, and uses Large Language Models to generate validated code fixes, deploying a fix only after explicit human approval.

Input

- Production application logs
- Git commit history
- Deployment records

AION

- Log analysis & incident detection
- Git & deployment correlation
- LLM root-cause analysis
- Patch generation & validation

Output

- Incident report & RCA
- Ranked suspect commits
- Validated code patch
- Human-approved, verified deployment

Motivation

Production incidents are among the most expensive moments in software engineering. When a service fails, engineers must search through thousands of log lines, work out which recent change caused the failure, write and test a fix, and push it through the deployment pipeline, often under severe time pressure. The tools for these steps are scattered: observability platforms stop at detection, while code repair and deployment remain manual. Recent advances in Large Language Models show that AI can explain failures and propose code fixes, but letting AI change production unchecked is unsafe. AION is motivated by the gap between these two worlds: automating the slow, repetitive investigation work while keeping deterministic validation and a human decision in front of every production deployment.

- **Slow, manual resolution**
- **Log and alert overload**
- **Fragmented tooling**
- **Unsafe automation**

Objectives of the Project

1. To detect production incidents automatically by collecting, normalizing and deduplicating application logs and flagging error spikes.
2. To identify the probable cause of each incident by correlating it with Git commits and deployment history.
3. To perform evidence-based root-cause analysis and generate candidate code patches using Large Language Models.
4. To validate every candidate patch through automated tests, staging deployment and replay of the failing requests.
5. To deploy a fix only after human approval of the exact validated commit, keeping a complete audit trail.

Literature Review

We reviewed 33 publications across the four research areas that AION brings together:

Log Parsing & Anomaly Detection

Turn raw logs into structured events and flag abnormal behaviour.

References: [2], [3], [4], [13], [14], [16], [17], [18], [19], [20], [21]

AIOps & Incident Management

Use AI and LLMs to diagnose and manage cloud incidents.

References: [1], [7], [10], [12], [15], [22], [23]

Root-Cause Localisation & Change Analysis

Locate the faulty service or the change that introduced a failure.

References: [5], [6], [24], [25], [26]

Automated Program Repair & LLM Agents

Generate and validate code fixes with learned models and LLM agents.

References: [8], [9], [11], [27], [28], [29], [30], [31], [32], [33]

Research gap: each area solves one stage in isolation. No reviewed work connects live log detection, change correlation, LLM diagnosis, validated repair and human-approved deployment in one workflow.

Literature Review: Log Analysis

Ref.	Paper & Authors	Venue (Year)	Contribution	Gap w.r.t. AION
[2]	Drain: Online Log Parsing with Fixed Depth Tree (P. He et al.)	IEEE ICWS (2017)	Online parser that groups log lines into templates using a fixed-depth tree.	Parsing only; no diagnosis or fix.
[3]	DeepLog: Anomaly Detection from System Logs (M. Du et al.)	ACM CCS (2017)	LSTM learns normal log sequences and flags deviations.	Detection only; needs training on normal logs.
[17]	A Survey on Automated Log Analysis for Reliability Engineering (S. He et al.)	ACM CSUR (2021)	Reviews 158 papers across logging, parsing, anomaly detection and diagnosis.	Survey; remediation out of scope.
[18]	Log-based Anomaly Detection Without Log Parsing (V.-H. Le, H. Zhang)	IEEE/ACM ASE (2021)	NeuralLog: semantic vectors + Transformer, F1 > 0.95 on public datasets.	Detection only; no root cause.
[21]	LILAC: Log Parsing using LLMs with Adaptive Parsing Cache (Z. Jiang et al.)	ACM FSE (2024)	LLM parsing with a template cache; large accuracy gain, far fewer LLM queries.	Parsing only.
[14]	Log Anomaly Detection in AIOps using LLMs (M. De la Cruz Cabello et al.)	Systems & Soft Computing (2026)	Locally run LogBERT detects anomalous syslog sequences in real time.	No root cause or fix.

Literature Review: AIOps & Root Cause

Ref.	Paper & Authors	Venue (Year)	Contribution	Gap w.r.t. AION
[1]	A Survey of AIOps Methods for Failure Management (P. Notaro et al.)	ACM TIST (2021)	Taxonomy of AIOps methods for failure prevention, detection, RCA and remediation.	Survey; stages studied separately.
[22]	Towards Intelligent Incident Management (Z. Chen et al.)	ESEC/FSE (2020)	Two-year study of Microsoft incident management; IcM BRAIN framework.	Proprietary; no code repair.
[23]	Recommending Root-Cause and Mitigation Steps using LLMs (T. Ahmed et al.)	ICSE (2023)	Evaluates GPT-3.x on 40,000+ Microsoft incidents.	Uses incident text only; no validated fix.
[7]	Automatic Root Cause Analysis via LLMs for Cloud Incidents (Y. Chen et al.)	EuroSys (2024)	LLM-based root-cause analysis for cloud incidents.	Diagnosis only.
[24]	Anomaly Detection and Failure RCA in (Micro)Service Applications (J. Soldani, A. Brogi)	ACM CSUR (2022)	Survey of anomaly detection and RCA in multi-service apps.	Survey; no remediation.
[26]	When Do Changes Induce Fixes? (J. Śliwerski et al.)	MSR (2005)	Links version history to bug reports to find fix-inducing changes.	Offline, after the fact.

Literature Review: Program Repair

Ref.	Paper & Authors	Venue (Year)	Contribution	Gap w.r.t. AION
[27]	Automated Program Repair (C. Le Goues, M. Pradel, A. Roychoudhury)	Comm. ACM (2019)	Review of search-, semantics- and learning-based repair.	Overview; no live telemetry.
[29]	Automated Program Repair in the Era of Large Pre-trained LMs (C. S. Xia et al.)	ICSE (2023)	Studies 9 models (125M–20B) on 5 datasets; LLMs beat earlier APR tools.	Benchmark bugs only.
[8]	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair (I. Bouzenia et al.)	ICSE (2025)	Agent fixes 164 Defects4J bugs using tools and tests.	~270K tokens per bug; benchmark bugs.
[30]	SWE-bench: Can LMs Resolve Real-World GitHub Issues? (C. E. Jimenez et al.)	ICLR (2024)	Benchmark of 2,294 real GitHub issues from 12 Python repos.	Issue-driven; no production signal.
[32]	AutoCodeRover: Autonomous Program Improvement (Y. Zhang et al.)	ISSTA (2024)	AST-aware code search + LLM; 19% on SWE-bench lite at ~$0.43 per issue.	No deployment or approval gate.
[11]	LLM for Automated Vulnerability Repair (U. Kulsum et al.)	ACM AIware (2024)	Reasoning and patch-validation feedback improve LLM fixes.	Vulnerabilities only; no deployment.

Feasibility Study

Technical Feasibility

- Built on mature open-source tools: Python, FastAPI, SQLite, React and Git.
- A working prototype already runs the full loop end to end, with 44 automated tests.
- LLM can be the Claude API or a local model via Ollama.

Economic Feasibility

- Runs on the team's existing laptops; no servers to buy.
- All software used is free and open source.
- Only optional cost: Claude API usage (about ₹2,500 budgeted).

Operational Feasibility

- Fits the existing Git workflow: fixes are normal commits on branches.
- Engineers stay in control through the approval gate and dashboard.
- Every action is recorded in an audit trail.

Schedule Feasibility

- Phased plan within AY 2026–27: design, build, integrate, evaluate, document.
- Core pipeline already complete; remaining work is hardening and evaluation.

Scope of the Project

In Scope

Log ingestion: file tailing and HTTP API for structured JSON logs

Incident detection: signatures and error-spike detection

Change correlation: ranking suspect commits from Git and deployments

Root-cause analysis: evidence-cited LLM diagnosis and incident report

Patch generation & validation: tests, staging and request replay

Human approval, deployment & audit: via a web dashboard

Out of Scope (Future Work)

- Languages other than Python (Java, Node.js)
- Kubernetes / ArgoCD and real CI (GitHub Actions) integration
- Metrics- and trace-based detection
- Authentication and role-based approvers
- Automatic rollback and canary deployments
- Container sandbox for patch validation

Requirement Analysis: Hardware

Component	Specification	Qty	Market Value (₹)	Project Cost (₹)
Development laptop	Intel Core i5 / AMD Ryzen 5, 16 GB RAM, 512 GB SSD	3	55,000 each	0 (owned)
GPU for local LLM (optional)	NVIDIA RTX 3050, 6 GB VRAM (in one laptop)	1	Included above	0 (owned)
Storage	20 GB free for code, database and local model files	—	—	0
Internet connection	Broadband / college Wi-Fi for GitHub and LLM API	—	—	0 (existing)
Display	1920 × 1080 or higher	3	Included above	0 (owned)
Total hardware			**≈ 1,65,000**	**0**



No dedicated server is needed. AION, the monitored demo service and its staging copy all run as local processes on one laptop. A local LLM needs the GPU laptop; with the Claude API, any of the three laptops is enough.

Requirement Analysis: Software

Software	Purpose	Licence	Cost (₹)
Python 3.10+, FastAPI, Uvicorn	Backend language, REST API and server	Open source	0
SQLite + SQLAlchemy 2.0	Database and ORM	Open source	0
React.js + Vite, Node.js 18+	Web dashboard and build tooling	Open source	0
Git & GitHub	Version control, history and blame	Free tier	0
pytest, httpx	Automated tests, patch validation, request replay	Open source	0
Ollama + qwen2.5-coder (7B)	Local LLM for offline / private analysis	Open source	0
VS Code, Windows / Linux	Development environment	Free	0
Claude API (Anthropic)	Cloud LLM for root-cause analysis and patches	Pay per use	≈ 2,500 (estimate)
Total software			**≈ 2,500**

Estimated total project cost: ≈ ₹2,500 (hardware already owned; ₹0 if only the local LLM is used)

Project Design: System Architecture

Closed-loop incident workflow:
AI investigates and proposes — a human approves before production.

Git Repository & Deployment History

LLM (Claude API or local model)

AION PLATFORM

Production Application
Live service emitting logs and errors

logs

1 Log Ingestion & Filtering
Collects, parses and filters application logs

2 Deduplication & Incident Detection
Groups repeated errors and flags error spikes

3 Git & Deployment Correlation
Ranks recent commits and releases as suspects

4 AI Root-Cause Analysis
Evidence-cited diagnosis, incident report & ticket

5 Candidate Patch Generation
LLM fix as a reviewable diff on its own branch

6 CI/CD & Staging Validation
Tests, staging deploy and traffic replay

7 Human Approval Gate
Engineer approves the exact validated commit

8 Production Deployment
Deploys the approved fix and verifies recovery

deploy

Engineer
approves or rejects via dashboard

Incident Records & Audit Trail
RCA reports · patches · validation results · approvals

Project Design: Mathematical Model

Notation

L = stream of application log events

s = error signature (masked template + stack)

$c_t(s)$ = occurrences of s in current window

b(s) = average occurrences per baseline window

C = recent commits, D = deployments

S(c) = suspicion score of commit c

E = evidence pack, R = root-cause report

P = candidate patch, A = human approval

Formulation

Signature: $s = h(\text{type}, \text{frames}, \text{mask}(\text{message}))$

Incident: $c_t(s) \geq m$ and $c_t(s) \geq \rho \cdot b(s)$

Suspect score: $S(c) = \min(1, \Sigma\, w_k \cdot f_k(c)) \cdot \lambda(c)$

Probable cause: $c^* = \arg\max S(c)$

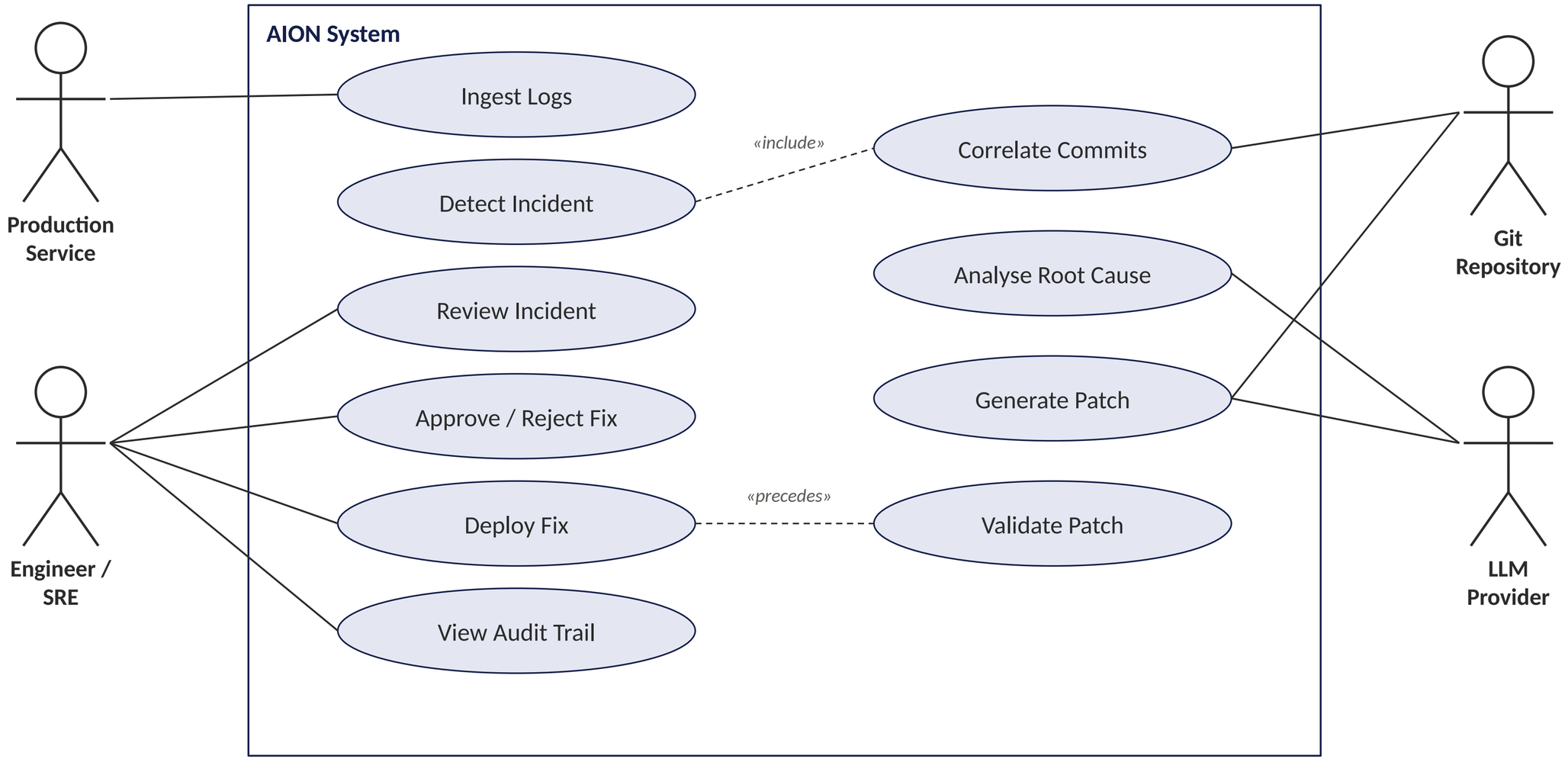
Validation: $V(P) = \text{syntax} \wedge \text{fail-to-pass} \wedge \text{tests} \wedge \text{staging} \wedge \text{replay}$

Deploy rule: $\text{deploy}(P) \Leftrightarrow V(P) = 1 \wedge A = \text{approve} \wedge \text{sha}(A) = \text{sha}(P)$

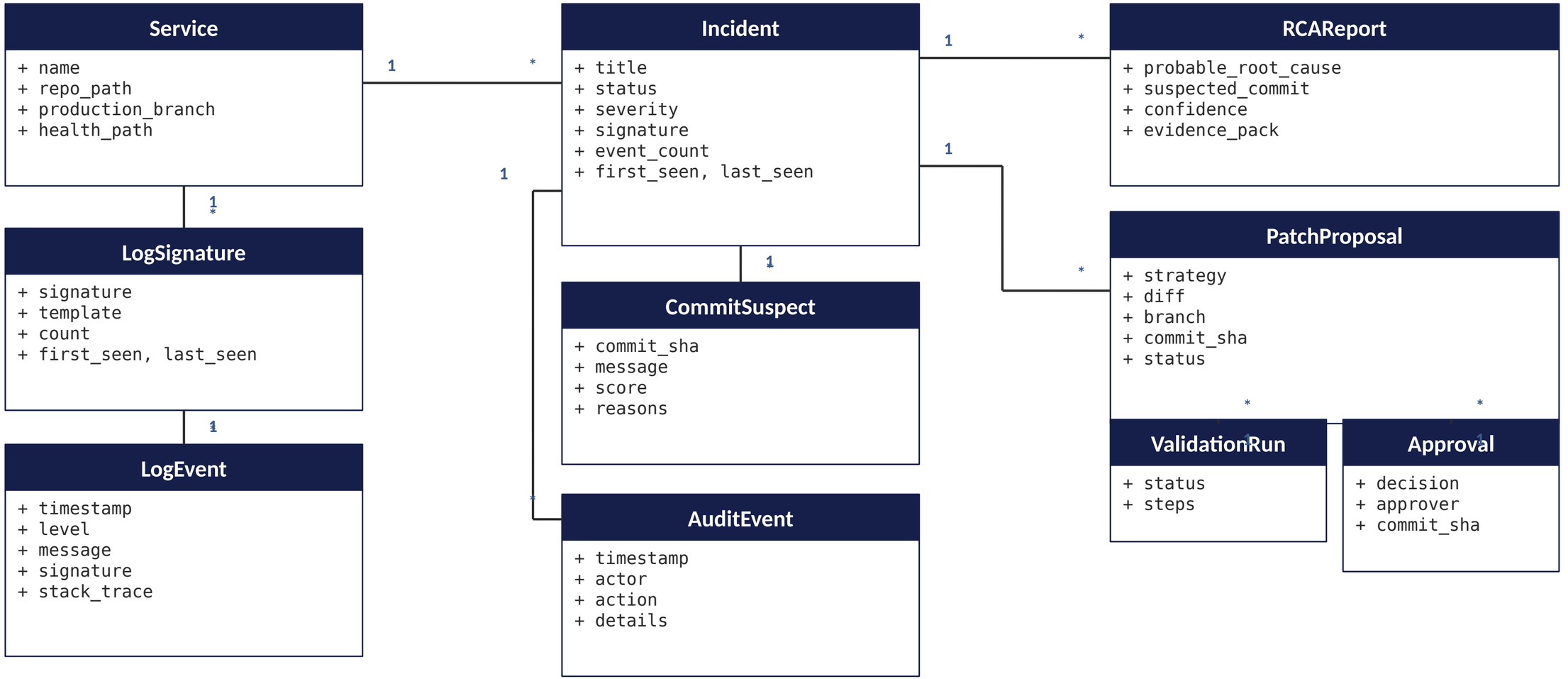
System output: $\text{AION}(L, C, D) \rightarrow \{ I, c^*, R, P, V, A \}$

Defaults: m = 5 errors, ρ = 3, window 60 s, baseline 30 min. Weights w: blame line 0.40, blame function 0.20, deployment 0.20, file 0.15, dependency 0.15, keyword 0.05; λ = 0.5 for code already stable before the incident. S(c) ranks suspects; it is not a probability.

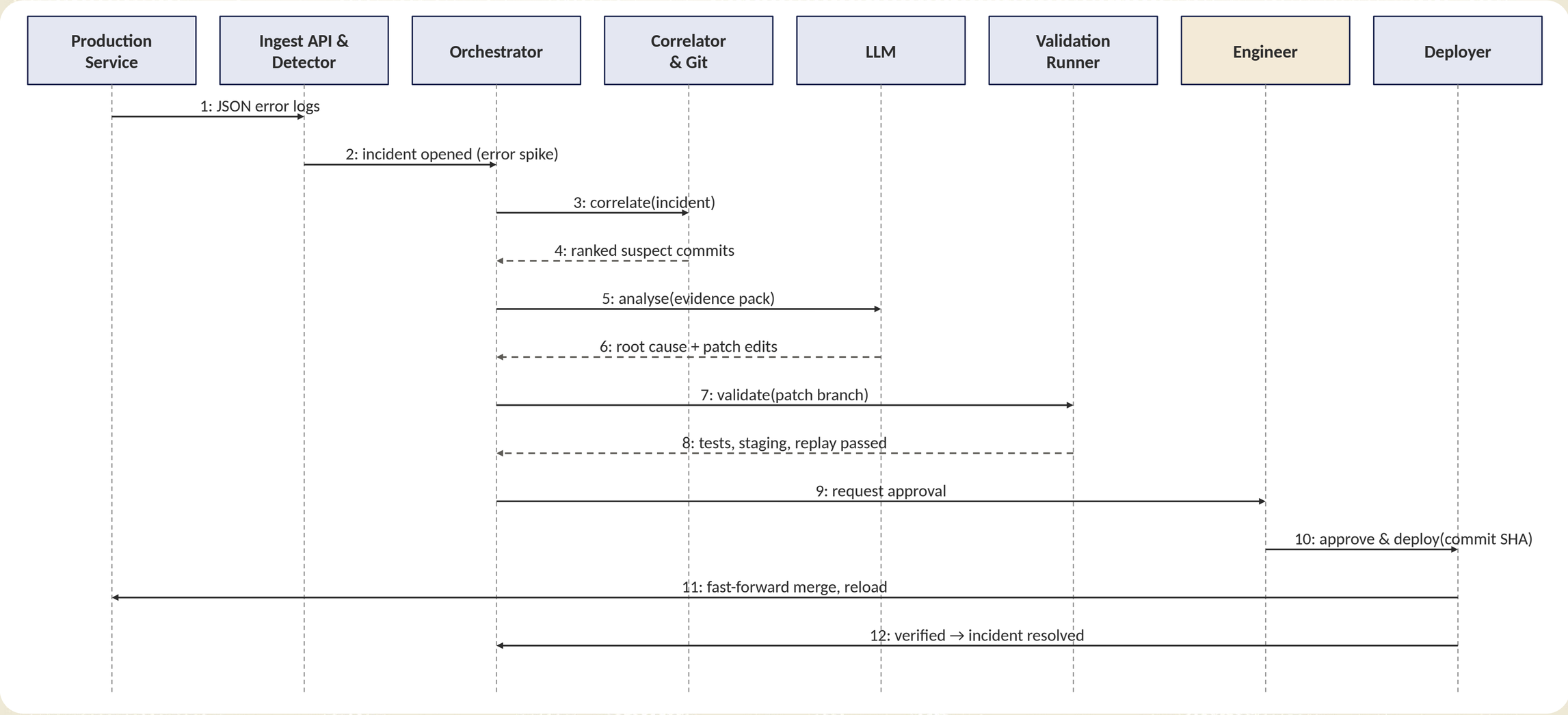
Project Design: Use Case Diagram



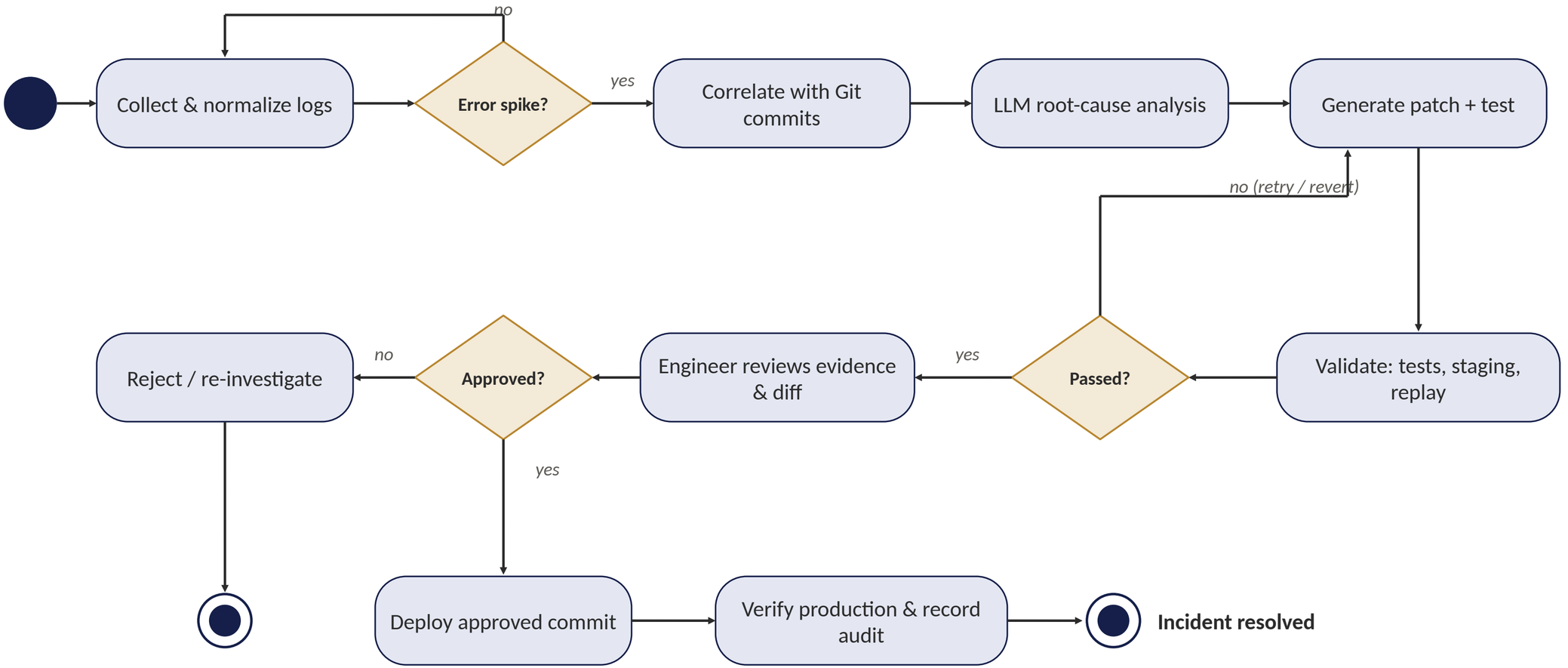
Project Design: Class Diagram



Project Design: Sequence Diagram



Project Design: Activity Diagram



References (1/3)

No.	Authors	Title	Venue / Year
[1]	P. Notaro, J. Cardoso, and M. Gerndt	A Survey of AIOps Methods for Failure Management	ACM TIST, vol. 12, no. 6, 2021
[2]	P. He, J. Zhu, Z. Zheng, and M. R. Lyu	Drain: An Online Log Parsing Approach with Fixed Depth Tree	IEEE ICWS, 2017
[3]	M. Du, F. Li, G. Zheng, and V. Srikumar	DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning	ACM CCS, 2017
[4]	W. Meng et al.	LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs	IJCAI, 2019
[5]	L. Wu, J. Tordsson, E. Elmroth, and O. Kao	MicroRCA: Root Cause Localization of Performance Issues in Microservices	IEEE/IFIP NOMS, 2020
[6]	T. Wittkopp, A. Acker, and O. Kao	Progressing from Anomaly Detection to Automated Log Labeling and Pioneering Root Cause Analysis	AIOps Workshop, IEEE ICDM, 2023
[7]	Y. Chen et al.	Automatic Root Cause Analysis via Large Language Models for Cloud Incidents	EuroSys, 2024
[8]	I. Bouzenia, P. Devanbu, and M. Pradel	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair	IEEE/ACM ICSE, 2025
[9]	Q. Zhang et al.	A Systematic Literature Review on Large Language Models for Automated Program Repair	arXiv:2405.01466, 2024
[10]	D. Roy et al.	Exploring LLM-based Agents for Root Cause Analysis	ACM FSE, 2024
[11]	U. Kulsum, H. Zhu, B. Xu, and M. d'Amorim	A Case Study of LLM for Automated Vulnerability Repair	ACM AIware, 2024

References (2/3)

No.	Authors	Title	Venue / Year
[12]	L. Zhang, T. Jia, M. Jia, et al.	A Survey of AIOps in the Era of Large Language Models	ACM Computing Surveys, 2026
[13]	Z. Ma, J. Yang, and T.-H. Chen	LLM4Log: A Systematic Review of Large Language Model-based Log Analysis	arXiv:2604.16359, 2026
[14]	M. De la Cruz Cabello, T. P. Sales, and M. R. Machado	Log Anomaly Detection in AIOps: A Real-World Implementation Using LLMs	Systems and Soft Computing, vol. 8, 2026
[15]	—	Integrating Large Language Models with Cloud-Native Observability for Automated Root Cause Analysis and Remediation	Intl. Conf. on AI, Systems and Network Security, 2025
[16]	J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu	Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics	IEEE ISSRE, 2023
[17]	S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu	A Survey on Automated Log Analysis for Reliability Engineering	ACM Computing Surveys, vol. 54, no. 6, 2021
[18]	V.-H. Le and H. Zhang	Log-based Anomaly Detection Without Log Parsing	IEEE/ACM ASE, 2021
[19]	H. Guo, S. Yuan, and X. Wu	LogBERT: Log Anomaly Detection via BERT	arXiv:2103.04475, 2021
[20]	Y. Lee, J. Kim, and P. Kang	LAnoBERT: System Log Anomaly Detection based on BERT Masked Language Model	arXiv:2111.09564, 2023
[21]	Z. Jiang et al.	LILAC: Log Parsing using LLMs with Adaptive Parsing Cache	ACM FSE, 2024
[22]	Z. Chen et al.	Towards Intelligent Incident Management: Why We Need It and How We Make It	ESEC/FSE, 2020

References (3/3)

No.	Authors	Title	Venue / Year
[23]	T. Ahmed et al.	Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models	ICSE, 2023
[24]	J. Soldani and A. Brogi	Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey	ACM Computing Surveys, vol. 55, no. 3, 2022
[25]	M. Ma et al.	AutoMAP: Diagnose Your Microservice-based Web Applications Automatically	The Web Conference (WWW), 2020
[26]	J. Śliwerski, T. Zimmermann, and A. Zeller	When Do Changes Induce Fixes?	MSR Workshop, 2005
[27]	C. Le Goues, M. Pradel, and A. Roychoudhury	Automated Program Repair	Communications of the ACM, vol. 62, no. 12, 2019
[28]	C. S. Xia and L. Zhang	Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-shot Learning	ESEC/FSE, 2022
[29]	C. S. Xia, Y. Wei, and L. Zhang	Automated Program Repair in the Era of Large Pre-trained Language Models	ICSE, 2023
[30]	C. E. Jimenez et al.	SWE-bench: Can Language Models Resolve Real-World GitHub Issues?	ICLR, 2024
[31]	J. Yang et al.	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	NeurIPS, 2024
[32]	Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury	AutoCodeRover: Autonomous Program Improvement	ACM ISSTA, 2024
[33]	P. Lewis et al.	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	NeurIPS, 2020